Approach and implementation

I extended my existing sign-analysis pass so that it can optionally turn its dataflow facts into executable assertions. The pass is still registered as `zero-analysis`, but it runs `sign::SignAnalysis`. I enabled the transformation with `signanalysischeck=true`; the default mode continues to print annotations without changing the program.

The implementation is a `signanalysischeck()` helper in `Plugin.cpp`, called from `runOnOperation()` after the dataflow solver reaches a fixed point. It walks LLVM function bodies, looks up each integer value's final `SignLattice`, and saves useful `(Value, Kind)` pairs in a `std::vector`. Missing states and `Top`/`Bottom` are skipped. Collecting facts before inserting checks avoids using temporary solver facts or changing the IR during analysis.

I insert a check immediately after an ordinary operation result is defined, or at the beginning of a block for a block argument. This also checks facts propagated through joins and loop backedges. Global initializers and terminator results are excluded; the latter would require checks on particular edges.

Turning facts into assertions

Each useful abstract fact becomes an `arith.cmpi` followed by `cf.assert`. Comparisons retain the original integer width and use signed ordering:

Inferred fact	Runtime condition
`One`	`x == 1`
`Zero`	`x == 0`
`Negative`	`x < 0`
`Positive`	`x > 0`
`NonPositive`	`x <= 0`
`NonNegative`	`x >= 0`

For example, a `NonNegative` fact on an 8-bit square produces the following check (SSA names and diagnostic text shortened for clarity):

```
%square = llvm.mul %small, %small : i8
%zero = arith.constant 0 : i8
%valid = arith.cmpi sge, %square, %zero : i8
cf.assert %valid, "expected nonnegative at report/reproducer.mlir:4:15"
```

The actual diagnostic includes the expected sign and the value's source location. I then lower `arith` and `cf` to the LLVM dialect, translate with `mlir-translate --mlir-to-llvmir`, and compile with clang at `-O0`. The lowered assertion prints a message and aborts when its condition is false.

Minimal reproducer

I manually minimized the square case from `test/runtime.mlir` to this complete file, `report/reproducer.mlir`; I did not need to run `llvm-reduce`:

```
module {
  llvm.func @square_i8(%x: i32) -> i32 {
    %small = llvm.trunc %x : i32 to i8
    %square = llvm.mul %small, %small : i8
    %result = llvm.sext %square : i8 to i32
    llvm.return %result : i32
  }
}
```

Failure and root cause

The small C driver in `report/driver.c` calls `square_i8(12)` and prints the returned value. It disables stdout buffering so the assertion message survives redirection before `abort()`.

The unmodified program returns **−112**. The checked program aborts at the multiply with this message (wrapped here for display):

```
sign-analysis assertion failed: expected nonnegative at
loc("report/reproducer.mlir":4:15)
```

The multiplication transfer function in `SignAnalysis.cpp` assigns `NonNegative` whenever both operands are the same SSA value. That rule follows the mathematical intuition that a square cannot be negative, but ignores fixed-width integer wraparound. For this input, $12 \times 12 = 144$; the 8-bit representation is `0x90`, which denotes **−112** when interpreted as signed. The inserted signed comparison therefore fails. The same issue occurs for `i32`: $46341 \times 46341 = 2147488281$, which wraps to **−2147479015**.

These `llvm.mul` operations have no overflow flags, so wraparound is defined LLVM behavior. This is a false analysis fact, rather than a test that executes overflow poison or relies on C signed-overflow behavior.

This was an unintended bug already in my repository. The square rule is present in the original committed `SignAnalysis.cpp`. I did not insert an unsound transfer function or modify the existing transfer rules to obtain a failure. They are retained so the counterexample remains reproducible. A repair would return `Top` when wraparound can invalidate a sign rule, unless sufficient no-overflow information is available. Other arithmetic sign rules need the same audit.

Testing experience and reproduction

I compared original and checked executables using every signed 8-bit input, 17 selected 32-bit boundary values, 1,000 seeded random 32-bit values, and branch/loop cases. The sweep ran **2,296 function/input cases**: **1,688 passed** and **608 triggered the false nonnegative claim**. Every baseline result matched a Python integer oracle. These are repeated counterexamples to the same bug, not 608 distinct bugs. The two CTest tests also passed, including a test that explicitly expects the known failures. Passing that test does not mean the analysis is sound.

From the repository root, with the plugin built in `build-runtime`:

```
BUILD_DIR=build-runtime sh signanalysischeck.sh \
  report/reproducer.mlir build-runtime/report/checked report/driver.c
build-runtime/report/checked

python3 check-runtime.py --build-dir build-runtime \
  --random-inputs 1000 --seed 0
```

The first executable aborts; the sweep exits with status 1 when it finds counterexamples and saves them in `build-runtime/runtime-check/report.json`. The main lesson was that a plausible integer-arithmetic rule needs to match LLVM's bit-vector semantics, not just the corresponding mathematical operation.